

Engineering Stable Behaviour in Self-Modifying LLM Agents

Brad Long*

The University of Melbourne
Melbourne, Australia
brad.long@student.unimelb.edu.au

Technology One Ltd
Brisbane, Australia
brad.long@technology1.com

25 April 2026

Abstract

Recent conceptual work questions whether self-modifying intelligent systems can maintain stable identity or goals over time. While these arguments are philosophically compelling, their implications are not readily actionable for practitioners building modern large language model (LLM) agents and autonomous systems. This paper translates those concerns into concrete system design principles. It is argued that identity instability in adaptive agents is not merely theoretical; it is already observable in production systems as behavioural drift, goal inconsistency, and emergent failure modes. A practical architecture is proposed that treats agents as evolving systems rather than stable entities, using enforceable invariants, layered control boundaries, and continuous verification pipelines to maintain behavioural reliability under change.

Keywords

Large language model agents, agentic AI, behavioural stability, runtime governance, invariant enforcement, adaptive systems

1 Introduction

The recent article *Deconstructing Superintelligence: Identity, Self-Modification and Différance* challenges foundational assumptions about intelligent systems, particularly the notions of stable identity and persistent goals under self-modification [19]. These critiques suggest that the concept of a continuous, unified “agent” may not be coherent in systems capable of recursive self-improvement.

Modern LLM-based agents are not static artifacts. They evolve continuously through prompt iteration, retrieval-augmented context, tool integration, and memory accumulation [28]. Even without explicit self-modification, these systems change their behaviour over time simply because

*This work was conducted independently in the author’s personal capacity. The views expressed are the author’s own and do not necessarily reflect the views of Technology One Ltd.

Preprint DOI: <https://doi.org/10.13140/RG.2.2.16153.17769>

their inputs and context change [20]. This creates a fundamental mismatch. There is a tendency to design and reason about agents as if they were stable software components, but in practice they behave more like adaptive systems whose behaviour shifts under changing conditions. This gap is not just theoretical—it shows up in production as subtle regressions, unexpected edge cases, and gradual loss of previously validated behaviour. The core issue can be stated simply:

Behavioural guarantees degrade as systems evolve.

Prior work on alignment assumes that goals can be specified and preserved over time [5, 21]. However, more recent perspectives, including work on learned optimization [12], suggest that internal behaviour can diverge even when external objectives remain fixed. To address this gap, a structured framework is proposed, the Invariant-Bounded Agent Alignment Model (IBAAM), which is formally introduced in Section 7. The framework provides a layered decomposition of agent systems into execution, adaptation, control, and observability components, enabling invariant-based reasoning about behavioural stability under continuous change. Instead of trying to preserve a stable “agent identity,” the focus of this paper is on how to maintain stable behaviour under change using the IBAAM framework. The primary contribution of this work is the introduction of IBAAM as a unifying abstraction for reasoning about stability in adaptive LLM agent systems, together with a corresponding architectural model and semi-formal formulation.

2 Related Work

Recent work on LLM-based agent systems has primarily focused on improving reasoning capability, tool usage, and task decomposition within language models. Frameworks such as *ReAct* [29] and *Toolformer* [23] demonstrate that large language models can be extended beyond static text generation to support interactive decision-making and external tool invocation. Alongside these developments, broader agent orchestration approaches have explored modular pipelines combining prompting, memory, and execution layers to improve reliability and task performance [28, 18, 24]. More recent work such as *Invariant Guardrails* [14] introduces runtime policy enforcement mechanisms that monitor agent trajectories, tool calls, and message flows against declarative invariants, reflecting a growing recognition that agentic systems require explicit governance and safety controls during execution rather than relying solely on prompt alignment or model training.

While these approaches significantly improve reasoning and tool use capabilities, they generally do not provide an explicit, system-level mechanism for constraining behaviour through enforceable invariants over state transitions. In particular, existing frameworks operate primarily at the level of prompting, orchestration, or external runtime mediation, rather than embedding governance directly into the agent execution model itself. This limits their ability to provide guarantees under continuous adaptation, where behavioural drift is expected rather than exceptional. Invariant Guardrails moves closer toward runtime governance by introducing declarative execution policies, but its architecture primarily acts as an external monitoring and mediation layer around existing agents. By contrast, IBAAM introduces a dedicated invariant-bounded execution architecture in which planning, execution, adaptation, and self-modification occur as governed state transitions within a replayable event-sourced system. A comparative summary of these approaches is provided in Table 1.

Framework	Core Idea	Strength	Limitation
ReAct (Yao et al., 2023)	Interleaves reasoning and acting within LLM prompts to improve task performance.	Improves multi-step reasoning and tool use in language models.	Does not explicitly enforce system-level constraints or invariants. Behaviour is prompt-dependent and not formally bounded.
Toolformer (Schick et al., 2023)	Models learn when and how to call external tools via self-supervision.	Enables scalable tool usage without manual annotation.	Tool usage is learned probabilistically; no explicit safety or control layer guarantees.
Invariant Guardrails	Applies declarative runtime policies to monitor and constrain agent trajectories, tool calls, and message flows.	Introduces explicit runtime governance and policy enforcement for agent systems.	Primarily operates as an external mediation layer rather than embedding governance directly into state transition semantics.
Traditional Agent Pipelines	Structured orchestration of LLM calls with external tools and memory.	Improves modularity and engineering control.	Control logic is typically ad-hoc and not formally separated from adaptive behaviour.
IBAAM (this work)	Defines agents as invariant-bounded systems with explicit separation between execution, adaptation, control, and observability layers.	Provides formal invariant enforcement, replayable execution semantics, and drift-aware stability model.	Requires explicit policy specification and runtime enforcement infrastructure.

Table 1: Comparison of IBAAM with prior LLM agent frameworks

3 Problem Statement

Modern LLM-based agents do not exhibit stable identity or persistent goals under system evolution. Their behaviour emerges from the interaction of multiple components—model weights, prompts, retrieved context, memory, and tool interfaces [28, 7, 23, 18]. Because these components are continuously updated or reinterpreted at runtime, both identity and goal structure become emergent properties rather than fixed system state.

This leads to a fundamental mismatch in how these systems are engineered. Agents are often treated as stable software entities, but in practice they behave as adaptive systems whose internal conditions shift continuously. As a result, behavioural guarantees degrade as the system evolves.

From this perspective, three **structural properties** follow [7, 20, 9]:

- Identity is emergent: system behaviour depends on a composition of mutable components, not a fixed identity.
- Goals are reconstructed: objectives are inferred at runtime rather than stored as persistent state.

- Behaviour is drift-prone: small modifications in prompts, memory, or tools propagate non-linearly over time.

These properties manifest directly in deployed systems as **observable failure modes** [2, 18]:

- Behavioural drift: output distributions change over time due to evolving prompts, tools, or context.
- Goal erosion: optimisation pressures gradually weaken original constraints or intent.
- Hidden state corruption: memory introduces stale, conflicting, or biased information.
- Emergent violations: tool composition produces unsafe multi-step behaviours not present in individual actions.

These issues indicate that instability is not an edge case but a structural consequence of adaptive system design.

4 Design Principles for Stable Agent Systems

No.	Principle	Notes
1	Treat agents as systems, not entities	Shift from anthropomorphic views to system-level dynamics & interactions.
2	Separate adaptation (flexible) from control (stable)	Prevent adaptive components from violating safety-critical constraints.
3	Prefer invariants over vague objectives	Objectives are underspecified; invariants enable enforceable guarantees.
4	Assume continuous behavioural drift	Treat distribution shift as the default, not an exception.
5	Design for interaction effects, not isolated components	Failures often arise from coupling effects between components.

Table 2: Core design principles

The key shift is to treat agents as systems rather than entities. An agent is a composition of interacting components: model, prompting logic, memory, tools, and execution policies. System failures arise from interactions between components rather than isolated faults [17, 28].

To manage this, we must separate what is allowed to change from what must remain stable. This leads to a layered architecture where adaptive behaviour is isolated from control mechanisms. Equally important is replacing vague objectives with explicit invariants that can be validated at runtime [3]. Finally, systems must be designed under the assumption that behaviour will continuously evolve.

5 Reference Architecture

5.1 Execution Layer

At the bottom is the execution layer. This is where the actual work happens: the LLM generates outputs, tools are invoked, and actions are performed. On its own, this layer is powerful but unsafe. It will do what it is asked, but it has no inherent concept of boundaries or constraints.

Observability Layer (logs, traces, replay)
Control Layer (policies, invariants)
Adaptive Layer (prompt, memory, strategy)
Execution Layer (LLM + tools)

Table 3: Layered architecture of an LLM-based agent system

5.2 Adaptive Layer

Above that sits the adaptive layer. This is where most of the system’s flexibility lives. It dynamically assembles prompts at runtime from templates, context, and retrieved memory, selects tools, and decides how to approach a task. This layer is intentionally allowed to evolve. It is where improvements happen, but also where drift originates. The key is not to eliminate change here, but to contain its effects.

5.3 Control Layer

The control layer is the most important part of the architecture. It acts as a boundary between what the system wants to do and what it is allowed to do. Every action flows through this layer. It checks permissions, enforces invariants, and ensures that behaviour stays within defined limits. For example, it can require confirmation before destructive actions, enforce audit logging, or block operations outside a defined scope.

This layer must not be freely modifiable by the agent itself. If the system can change its own constraints, then those constraints lose meaning. This idea aligns with broader lessons from safety-critical systems and ML infrastructure, where strict boundaries are used to manage complexity and risk [24].

5.4 Observability Layer

At the top sits the observability layer. Without visibility, drift is invisible. By the time a failure is noticed, the system may have already diverged significantly from its intended behaviour. Observability provides the ability to trace decisions, replay scenarios, and detect anomalies. In practice, this means logging inputs, context, decisions, and outputs in a way that allows reconstruction of behaviour.

6 Invariants as the Basis of Agent Identity

In this work, an agent is defined not as a fixed policy or model instance, but as an equivalence class of system states preserved under a set of invariants. Let S_t denote the system state at time t , and let $I = \{i_1, \dots, i_n\}$ denote a set of invariant predicates over system state. An agent identity is then the set of all states satisfying:

$$A \equiv \{S_t \mid \forall i \in I, i(S_t) = \text{true}\}.$$

Under this view, identity is not determined by behaviour or capability, but by the preservation of constraints under transformation, including self-modification.

State transitions are governed by a function $T : S_t \rightarrow S_{t+1}$, which is valid only if invariants are preserved:

$$I(S_t) = \text{true} \wedge I(S_{t+1}) = \text{true}.$$

For self-modifying agents, this introduces a second-order requirement: not only must state transitions preserve invariants, but modifications to the system that define invariants must themselves be constrained by higher-order meta-invariants.

Examples of Invariants To make this concrete, it is helpful to distinguish between several classes of invariants relevant to agentic systems. These categories do not come from a single prior framework, but bring together common types of constraints seen across AI safety, control systems, and ML infrastructure literature. [2, 24, 6, 3, 21].

1. **Safety invariants** These constrain external action space.

- Access control invariant: The agent may only access resources explicitly authorised by its execution environment.
- Harm constraint invariant: Actions that lead to irreversible external harm are disallowed unless explicitly escalated to a higher authority layer.

These invariants define a hard boundary on action space, independent of internal reasoning changes.

2. **Integrity invariants** These constrain correctness and traceability of internal operation.

- Auditability invariant: All externally relevant actions must be traceable to a logged decision path.
- State consistency invariant: The system may not hold conflicting active policies without explicit resolution.

These ensure that system evolution remains interpretable and diagnostically recoverable.

3. **Identity-preserving invariants** These define continuity of the agent across updates.

- Version continuity invariant: Every system state must map to a versioned history of modifications.
- Behavioural drift bound: Policy updates must remain within a bounded divergence from a validated baseline unless re-certified.

These invariants ensure that “the same agent” remains identifiable despite internal evolution.

4. **Self-modification invariants** These constrain how the system may change itself.

- Scoped modification invariant: Changes are restricted to predefined domains (e.g. memory, strategy, prompts) unless explicitly escalated.
- Rollback invariant: Every modification must preserve the ability to revert to a prior valid state.
- Meta-invariant constraint: Rules governing modification of invariants themselves require higher-order approval.

These are critical in preventing recursive weakening of system constraints.

Under this formulation, agent identity is not a property of output behaviour, but of constraint stability under transformation. Capability expansion increases the reachable state space, but identity is preserved as long as transitions remain within the invariant-defined equivalence class. Failure modes can therefore be characterised not only as violations of invariants, but also as drift in invariant enforcement or corruption of the constraint system itself, where the guarantees specified by the invariants are weakened or altered over time.

7 Invariant-Bounded Agent Alignment Model (IBAAM)

The Invariant-Bounded Agent Alignment Model (IBAAM) is a structured conceptual framework for reasoning about LLM-based agents undergoing continuous adaptation. It is based on the premise that stability in adaptive systems should be defined by whether invariants are preserved as the system evolves over time.

IBAAM does not assume that agent internals are static or fully observable. Instead, it models agents as evolving systems whose behaviour is governed by a separation between adaptive processes and externally enforceable constraints.

7.1 System state decomposition

At discrete time t , an agent system is represented as:

$$S_t = (E_t, D_t, O_t) \quad (1)$$

where:

- E_t : execution subsystem (language model inference, tool invocation, and action execution)
- $D_t \in \mathcal{D}$: adaptive subsystem state (e.g., memory, prompts, learned strategies, retrieval state)
- O_t : observation state (logs, traces, and externally visible outputs)

This decomposition is not meant to be exhaustive or unique; instead, it separates action generation, behavioural adaptation, and post-hoc analysis. No assumption is made that D_t is directly observable or stable across time.

7.2 Invariant specification

Alignment constraints are defined independently of system state as a fixed set of predicates:

$$C = \{\phi_1, \phi_2, \dots, \phi_n\} \quad (2)$$

Each invariant is a predicate over observable system behaviour:

$$\phi_i : \mathcal{O} \rightarrow \{\text{true}, \text{false}\} \quad (3)$$

or more generally over an observable projection of system state:

$$\phi_i : \pi(S_t) \rightarrow \{\text{true}, \text{false}\} \quad (4)$$

where π denotes a (possibly partial) observation or evaluation function.

This formulation explicitly allows invariants to operate over incomplete or abstracted system representations, reflecting real-world observability constraints.

7.3 Control operator

IBAAM introduces a control operator Π_C that enforces invariants over candidate outputs. In practice, this operator represents a policy enforcement mechanism rather than a single deterministic function.

$$\Pi_C : \mathcal{O} \rightarrow \mathcal{O} \quad (5)$$

such that:

$$\Pi_C(o) = \begin{cases} o' & \text{if } \text{Validate}_C(o) = \text{true} \\ \text{Reject}(o) & \text{otherwise} \end{cases} \quad (6)$$

where:

- Validate_C denotes an approximate or probabilistic evaluation of invariants
- o' may be identical to o or a transformed (e.g. repaired or sanitised) output
- Reject denotes blocking, escalation, or safe fallback behaviour

This generalisation reflects that real-world enforcement mechanisms may include transformation and escalation pathways rather than strict binary rejection.

7.4 System evolution

The system evolves through interaction between adaptation, execution, control, and observation:

$$A_t = f_{\text{adapt}}(D_t, I_t) \quad (7)$$

$$E'_t = f_{\text{exec}}(E_t, A_t) \quad (8)$$

$$O_t = f_{\text{observe}}(E'_t, A_t) \quad (9)$$

$$O_t^C = \Pi_C(O_t) \quad (10)$$

$$D_{t+1} = g(D_t, O_t^C, I_t) \quad (11)$$

where A_t represents proposed actions or strategies and I_t denotes external inputs. This decomposition separates:

- **adaptation:** generation of candidate behaviour
- **execution:** realisation of behaviour
- **control:** constraint enforcement
- **update:** incorporation of outcomes into adaptive state

Control is applied both before execution to constrain admissible actions and after execution to validate observed outcomes prior to adaptive state updates. Invariants therefore govern both action selection and state assimilation, ensuring that only transitions consistent with defined constraints are permitted to influence system evolution.

7.5 Drift

Behavioural drift is defined as changes in the adaptive subsystem state over time:

$$\text{Drift}(t) = d(D_t, D_{t-k}) \quad (12)$$

where d is a task-specific measure of difference over $\mathcal{D}$, and k defines the comparison horizon.

No assumption is made that d is unique, Euclidean, or fixed; it may represent semantic, functional, or distributional divergence depending on representation. Drift is only meaningful in practice if it is associated with a higher likelihood of invariant violations under Π_C .

7.6 Stability

A system's behaviour over time is considered stable under IBAAM if all control-filtered outputs satisfy invariants over a horizon T .

$$\text{Stable}(S_{0:T}) \iff \forall t \in [0, T], \forall \phi \in C : \phi(O_t^C) = \text{true} \quad (13)$$

This definition decouples stability from identity preservation, fixed representations, or assumptions of stable behaviour over time, and instead defines it as consistent satisfaction of constraints over observable behaviour.

7.7 Control regimes

IBAAM distinguishes three regimes of constraint enforcement: the **uncontrolled regime**, in which invariants are not evaluated at runtime; the **advisory regime**, in which invariants are evaluated but not enforced; and the **enforced regime**, in which invariant violations trigger rejection, transformation, or escalation.

Only the enforced regime provides formal behavioural guarantees in the sense of invariant preservation under system evolution.

7.8 Limitations

IBAAM is a modelling framework rather than a complete implementation. Its guarantees depend on several assumptions:

1. **Control integrity:** The control operator Π_C is implemented in a trusted layer that is not modifiable by the adaptive subsystem. If this separation fails, invariants may not be enforceable.
2. **Observability constraints:** Invariants operate over observable or evaluable projections of system state. Incomplete observability may allow undetected violations.
3. **Specification quality:** The framework assumes invariants are meaningful and correctly specified. It does not address invariant discovery or correctness.
4. **Approximate evaluation:** In practice, invariant validation may be probabilistic or heuristic. Enforcement should therefore be interpreted as risk reduction rather than absolute guarantees.

These limitations imply that IBAAM should be interpreted as a structured constraint architecture for adaptive systems, rather than a complete solution to alignment.

8 Towards Practical Alignment in Adaptive Systems

This section operationalises the IBAAM framework across runtime control, system evolution, and implementation.

8.1 Continuous Alignment Framework

Traditional evaluation assumes that once a system passes tests, it is “good enough.” This assumption does not hold for adaptive systems [2, 24]. In such systems, behaviour is a trajectory rather than a fixed property, and therefore must be continuously validated. Continuous alignment treats correctness as an ongoing property of system execution. The goal is not certification, but sustained constraint satisfaction under changing conditions [8]. This requires repeated evaluation across both synthetic and real-world contexts.

Two complementary testing modes are required:

- **Regression evaluation:** ensures previously validated behaviours remain consistent under system updates.
- **Scenario evaluation:** tests behaviour across realistic task distributions and interaction patterns.

At runtime, alignment is enforced through an active control loop. Each candidate action is evaluated against policy constraints prior to execution [11]. This introduces a gating mechanism that operates continuously during system operation rather than at deployment time. Rather than assuming stability, this framework assumes continuous variation in system behaviour and focuses on ensuring that all observed behaviour remains within acceptable bounds defined by policy.

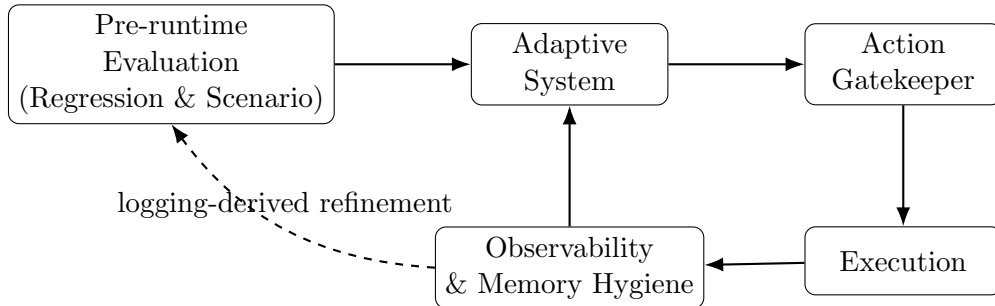


Figure 1: Compact continuous alignment loop integrating pre-runtime evaluation, runtime enforcement, and post-execution memory and observability feedback.

The layered architecture and control-loop diagram represent complementary views of the same system: the former describes the structural organisation of system components, while the latter captures runtime control dynamics and information flow during execution.

8.2 Managing Self-Modification

Self-modification refers to changes in the internal structure of the system over time, including updates to prompts, memory systems, tools, and behavioural policies. Unlike runtime alignment, which governs execution, this governs how the system itself evolves.

The key design requirement is separation of mutable and immutable components. Adaptive subsystems may evolve, but safety-critical components such as access control, execution constraints, and permission boundaries must remain fixed.

To make evolution reliable, changes must be explicitly versioned. Each modification produces a traceable system state that can be reproduced and compared over time [6, 16]. This enables systematic rollback and debugging of behavioural regressions introduced by updates.

Table 4 formalises these design constraints, providing a structured view of mutable and immutable components alongside the mechanisms governing versioning and deployment control. This representation serves as a reference model for implementing governed self-modification in adaptive systems.

System evolution should also be controlled through staged introduction of changes. High-impact modifications require additional validation steps such as human review, automated policy checks, and gradual rollout. Techniques such as canary deployment allow changes to be evaluated under limited exposure before full integration [13, 4].

Category	Control Constraint
Mutable components	Prompts, memory systems, strategies, and tool configurations may evolve under controlled updates governed by explicit validation and versioning rules.
Immutable components	Safety constraints, permission boundaries, and execution policies remain fixed across system versions to preserve invariant guarantees.
Version control	All system modifications are explicitly versioned to ensure reproducibility, auditability, and rollback capability.
Deployment control	High-impact changes require staged rollout, validation, and approval mechanisms prior to full integration.

Table 4: Control constraints for governed evolution of adaptive LLM systems under IBAAM.

8.3 Practical Implementation Patterns

Continuous alignment and self-modification governance can be directly mapped to established engineering mechanisms in production systems. The patterns below represent concrete instantiations of runtime control, observability, and controlled system evolution.

Action gatekeeper implements a runtime enforcement model by validating every external action against policy constraints prior to execution. This provides a deterministic control point between decision and action, consistent with the reference monitor concept and complete mediation principle in secure system design [1, 22].

Replayable execution supports observability and auditability by enabling full reconstruction of system behaviour from recorded inputs, context, and outputs. This aligns with distributed tracing and event logging systems, where execution can be reconstructed from recorded traces or event streams for debugging and analysis [25, 15].

Canary deployments operationalise the self-modification governance model by enabling controlled exposure of system changes prior to full integration. This allows behavioural impact to be evaluated under limited rollout conditions, as established in continuous delivery and progressive delivery practices [13, 4].

Memory hygiene mitigates long-term state degradation through structured expiration, validation, and conflict resolution mechanisms. This prevents uncontrolled accumulation of outdated or

inconsistent information in adaptive subsystems, reflecting principles from event-sourced system design and distributed state consistency management [15, 27].

These patterns are not novel in isolation; they are well-established in classical software engineering and MLOps systems. Their contribution here is the systematic mapping of these mechanisms onto adaptive LLM agent architectures with explicit separation of runtime control, system evolution, and observability.

Taken together, these mechanisms provide a bridge between the abstract constraints of IBAAM and concrete system design. The preceding patterns can be understood as practical instantiations of runtime control, system evolution governance, and observability in adaptive LLM agents.

9 Conclusion

Self-modifying AI systems challenge assumptions about identity and stability. These challenges are already visible in modern LLM-based systems. These observations suggest that the objective is not to prevent change, but to design systems in which change remains bounded by enforceable constraints. Stability does not come from preserving identity—it comes from enforcing invariants over system behaviour. The formal model makes this precise: identity may drift, but invariants define acceptable behaviour.

By separating adaptive behaviour from control, enforcing invariants, and continuously validating outcomes, we can build systems that remain reliable even as they evolve. The IBAAM framework provides a concrete way to operationalise this perspective, enabling reasoning about stability as invariant preservation under continuous adaptation, and reframes alignment as a runtime systems problem, not a training problem. IBAAM does not eliminate instability; it bounds its impact.

References

- [1] Anderson, J. P. (1972). *Computer Security Technology Planning Study*. Technical Report, James P. Anderson Co. <https://csrc.nist.gov/publications/detail/other/its/ir/1972/final/final-report-computer-security-technology-planning-study>
- [2] Amodei, D., Olah, C., Steinhardt, J., Christiano, P., Schulman, J., and Mané, D. (2016). *Concrete Problems in AI Safety*. arXiv preprint arXiv:1606.06565. <https://arxiv.org/abs/1606.06565>
- [3] Bai, Y., Kadavath, S., Kundu, S., Askell, A., Kernion, J., Jones, A., Chen, A., Goldie, A., Mirhoseini, A., McCandlish, S., et al. (2023). *Constitutional AI: Harmlessness from AI Feedback*. arXiv preprint arXiv:2212.08073. <https://arxiv.org/abs/2212.08073>
- [4] Beyer, B., Jones, C., Petoff, J., and Murphy, N. R. *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, 2016.
- [5] Bostrom, N. (2014). *Superintelligence: Paths, Dangers, Strategies*. Oxford University Press.
- [6] Breck, E., Cai, S., Nielsen, E., Salib, M., & Sculley, D. (2017). *The ML Test Score: A Rubric for ML Production Readiness*. Google Research Technical Report. <https://research.google/pubs/pub46555/>

- [7] Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. (2020). *Language Models are Few-Shot Learners*. In Advances in Neural Information Processing Systems (NeurIPS 2020). arXiv preprint arXiv:2005.14165. <https://arxiv.org/abs/2005.14165>
- [8] Christiano, P. F., Leike, J., Brown, T. B., Martic, M., Legg, S., and Amodei, D. *Deep Reinforcement Learning from Human Preferences*. Advances in Neural Information Processing Systems (NeurIPS), 2017.
- [9] Du, P., et al. (2026). *Memory for Autonomous LLM Agents: Mechanisms, Evaluation, and Emerging Frontiers*. arXiv preprint arXiv:2603.07670. <https://arxiv.org/abs/2603.07670>
- [10] Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., and Bouchachia, A. *A Survey on Concept Drift Adaptation*. ACM Computing Surveys, 46(4), 2014.
- [11] Hadfield-Menell, D., Russell, S., Abbeel, P., and Dragan, A. *The Off-Switch Game*. In Proceedings of the 26th International Joint Conference on Artificial Intelligence (IJCAI), 2017.
- [12] Hubinger, E., et al. (2019). *Risks from Learned Optimization in Advanced Machine Learning Systems*. arXiv preprint arXiv:1906.01820. <https://arxiv.org/abs/1906.01820>
- [13] Humble, J. and Farley, D. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley Professional, 2010.
- [14] Invariant Labs. *Guardrails*. Available at: <https://invariantlabs.ai/blog/guardrails> Accessed: May 16, 2026.
- [15] Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O’Reilly Media. <https://dataintensive.net/>
- [16] Kreuzberger, D., Kühn, N., and Hirschl, S. *MLOps: A Survey of Machine Learning Operations*. arXiv preprint arXiv:2303.07510, 2023.
- [17] Leveson, N. *Engineering a Safer World: Systems Thinking Applied to Safety*. MIT Press, 2011.
- [18] Park, J. S., O’Brien, J., Cai, C. J., Morris, M. R., Liang, P., and Bernstein, M. S. (2023). *Generative Agents: Interactive Simulacra of Human Behavior*. arXiv preprint arXiv:2304.03442. <https://arxiv.org/abs/2304.03442>
- [19] Perrier, E. (2026). *Deconstructing Superintelligence: Identity, Self-Modification and Différance*. arXiv preprint arXiv:2604.19845. <https://arxiv.org/abs/2604.19845>
- [20] Rath, A. (2026). *Agent Drift: Quantifying Behavioral Degradation in Multi-Agent LLM Systems Over Extended Interactions*. arXiv preprint arXiv:2601.04170. <https://arxiv.org/abs/2601.04170>
- [21] Russell, S. (2022). *Artificial Intelligence and the Problem of Control*. In Artificial Intelligence: A Modern Approach (or related volume context), pp. 19–24. Springer. https://doi.org/10.1007/978-3-030-86144-5_3
- [22] Saltzer, J. H. and Schroeder, M. D. (1975). *The Protection of Information in Computer Systems*. Proceedings of the IEEE, 63(9), 1278–1308. <https://doi.org/10.1109/PROC.1975.9939>

- [23] Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., and Scialom, T. (2023). *Toolformer: Language Models Can Teach Themselves to Use Tools*. arXiv preprint arXiv:2302.04761. <https://arxiv.org/abs/2302.04761>
- [24] Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J. F., and Dennison, D. (2015). *Hidden Technical Debt in Machine Learning Systems*. In Advances in Neural Information Processing Systems (NeurIPS 2015), pp. 2503–2511. <https://dl.acm.org/doi/10.5555/2969442.2969519>
- [25] Sigelman, B. H., Barroso, L. A., Burrows, M., Stephenson, P., Plakal, M., Beaver, D., Jaspan, S., and Shanbhag, C. (2010). *Dapper, a Large-Scale Distributed Systems Tracing Infrastructure*. Google Technical Report. <https://research.google/pubs/pub36356/>
- [26] Tallam, K. (2026). *Layered Mutability: Continuity and Governance in Persistent Self-Modifying Agents*. arXiv preprint arXiv:2604.14717. <https://arxiv.org/abs/2604.14717>
- [27] Vogels, W. (2009). *Eventually Consistent*. Communications of the ACM, 52(1), 40–44. <https://doi.org/10.1145/1435417.1435432>
- [28] Wang, L., et al. (2023). *A Survey of Large Language Model based Autonomous Agents*. arXiv preprint arXiv:2308.11432. <https://arxiv.org/abs/2308.11432>
- [29] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. (2023). *ReAct: Synergizing Reasoning and Acting in Language Models*. arXiv preprint arXiv:2210.03629. <https://arxiv.org/abs/2210.03629>